



Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

Artificial Intelligence

Second Semester 2025-2026

Project Two Report

Prepared by:

Student Name: Lara Daifallah

ID: 1230239

Instructor: Dr. Yazan Abufarha

Section: 1

Date: 25/6/2026

Introduction

This project presents the implementation and evaluation of machine learning classification methods covered in the ENCS3340 course using Python and Scikit-learn. The work is divided into three parts. In Part A, the ID3 algorithm is implemented from scratch and applied to the PlayTennis dataset in order to compute entropy, information gain, and construct the same multi-way decision tree presented in the lecture slides. In Part B, Decision Tree and Naive Bayes are compared on the same dataset using Leave-One-Out Cross-Validation, which is more suitable for very small datasets. In Part C, a Decision Tree classifier is applied to the UCI Iris dataset to evaluate performance on a larger dataset with continuous features using accuracy, confusion matrix, precision, recall, and F1-score.

Overall, the project connects the theoretical concepts studied in the course with practical implementation and analysis. It highlights how decision trees can be built and interpreted, how Naive Bayes performs on categorical data, and how classification models can be evaluated on both small textbook examples and real benchmark datasets.

Part A - Decision Tree Classification

First : This is our data description:

Feature	Type	Values
Outlook	Categorical	Sunny, Overcast, Rain
Temperature	Categorical	Hot, Mild, Cool
Humidity	Categorical	High, Normal
Wind	Categorical	Weak, Strong
PlayTennis	Target	Yes (9 instances), No (5 instances)

The ID3 algorithm was implemented entirely in Python without using **sklearn** tree functions during the learning stage. The implementation follows the same procedure presented in the lecture slides: first computing the entropy of the current subset, then evaluating the information gain of each remaining attribute, selecting the attribute with the highest gain as the split node, and recursively repeating the process for each resulting subset. The recursion stops when all instances in a subset belong to the same class or when no attributes remain.

Unlike sklearn's DecisionTreeClassifier, which builds binary splits on encoded values, this implementation performs true multi-way splits by creating one branch for each distinct attribute value. As a result, the learned tree structure matches the decision tree shown in the lecture slides.

Information Gain Analysis

The total entropy of the full PlayTennis dataset (9 Yes,5 No) is:

$$I(9/14, 5/14) = -(9/14) \log_2(9/14) - (5/14) \log_2(5/14) = 0.940 \text{ bits}$$

Information gain was then computed for each attribute.

Outlook achieved the highest information gain (0.247 bits), so it was selected as the root node. The Overcast branch becomes a pure leaf immediately because all four corresponding instances belong to class Yes. The Sunny subset is then best split by Humidity, while the Rain subset is best split by Wind. As a result, the final ID3 tree uses only three of the four available attributes and classifies all training instances correctly.

Note : All entropies are shown in the run.

Attribute	Gain(S, A)	Slide Value	Match
Outlook	0.247	0.246	Yes
Humidity	0.152	0.151	Yes
Wind	0.048	0.048	Yes
Temperature	0.029	0.029	Yes

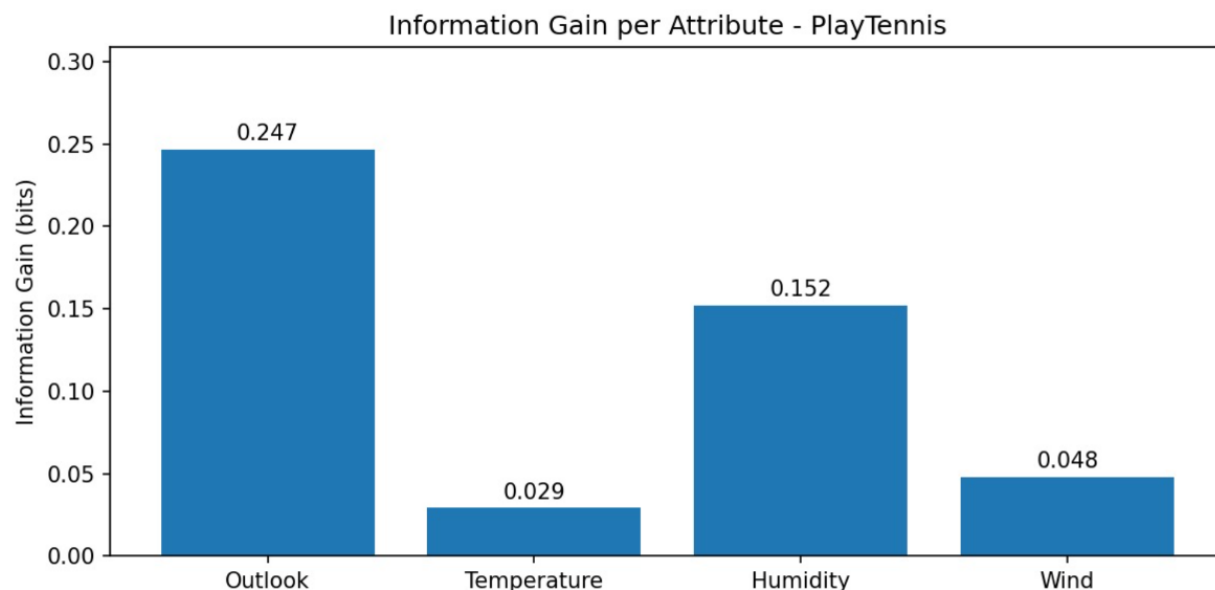
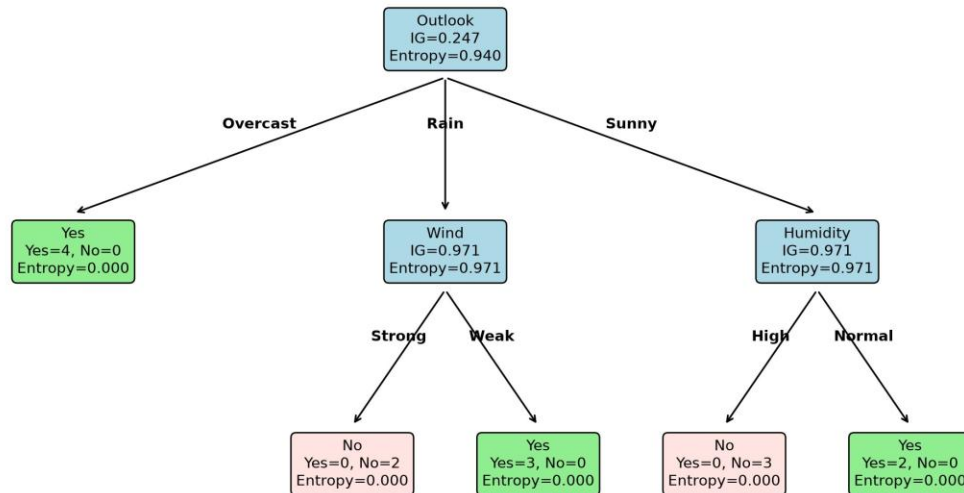


Figure 1 Information gain per attribute. Outlook is selected to be the root

Decision Tree

ID3 Decision Tree - PlayTennis Dataset



Part B - Classifier Comparison (Decision Tree vs Naive Bayes)

Leave-One-Out Cross-Validation:

The PlayTennis dataset contains only 14 instances, so a fixed train/test split would produce a very small test set and an unstable accuracy estimate. For this reason, Leave-One-Out Cross-Validation (LOO-CV) was used, as recommended in the lectures for very small datasets. In LOO-CV, the model is trained 14 times: in each iteration, one instance is used for testing and the remaining 13 are used for training. Each instance is therefore tested exactly once, and the final accuracy is computed over all 14 predictions.

Classifiers :

First is a **Decision Tree** trained with the **entropy** criterion, so that its split selection follows the same information-gain principle used by ID3 in the lecture slides. In each LOO fold, the tree was trained on 13 instances and used to predict the one held-out example.

The second classifier was **Naive Bayes**, implemented using CategoricalNB with **alpha = 0**, meaning that no Laplace smoothing was applied. This setting was chosen so that the estimated probabilities match our slides calculations exactly so I have a correct reference, where all probabilities are computed directly from raw frequency counts. As a result, the Naive Bayes calculations in this part follow the same arithmetic convention used in the slides.

Naive Bayes Probability Tables (Full Dataset)

The Naive Bayes prior and conditional probability tables were computed on the full 14-instance PlayTennis dataset. These tables are used to illustrate how Naive Bayes estimates class probabilities from frequency counts before comparing its predictive performance against the Decision Tree under LOO-CV.

Class	Count	P(class)
Play = Yes	9/14	0.6429
Play = No	5/14	0.3571
Outlook	P(. Yes)	P(. No)
Overcast	4/9=0.4444	0/5=0.0
Rain	3/9=0.3333	2/5=0.4
Sunny	2/9=0.2222	3/5=0.6
Temperature	P(. Yes)	P(. No)
Cool	3/9=0.3333	1/5=0.2
Hot	2/9=0.2222	2/5=0.4
Mild	4/9=0.4444	2/5=0.4
Humidity	P(. Yes)	P(. No)
High	3/9=0.3333	4/5=0.8
Normal	6/9=0.6667	1/5=0.2
Wind	P(. Yes)	P(. No)
Strong	3/9=0.3333	3/5=0.6
Weak	6/9=0.6667	2/5=0.4

Naive Bayes Test Example

$x' = (\text{Outlook} = \text{Sunny}, \text{Temperature} = \text{Cool}, \text{Humidity} = \text{High}, \text{Wind} = \text{Strong}).$

Using the probability tables from the full PlayTennis dataset:

$$P(\text{Yes}|x') \sim (9/14) * (2/9) * (3/9) * (3/9) * (3/9) = 0.0053$$

$$P(\text{No}|x') \sim (5/14) * (3/5) * (1/5) * (4/5) * (3/5) = 0.0206$$

Since $P(\text{No}|x') > P(\text{Yes}|x')$, the predicted class is **No**. This manual calculation verifies that the implemented probability tables reproduce the same Naive Bayes arithmetic presented in the slides.

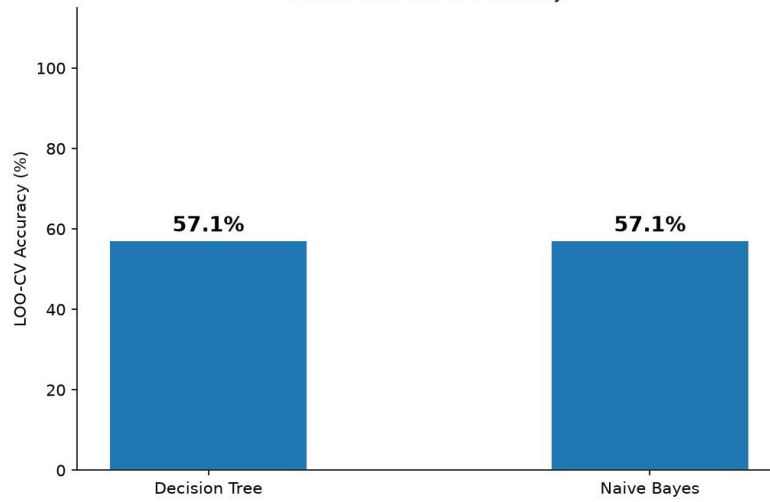
LOO-CV Results

Under Leave-One-Out Cross-Validation, both classifiers correctly classified 8 out of 14 instances, giving an identical accuracy of **57.1%**. However, this tie does not mean that the two models behave in the same way. The per-instance prediction results show that the Decision Tree and Naive Bayes make different errors on different examples, reflecting the difference between rule-based splitting in decision trees and probabilistic prediction in Naive Bayes.

Per-instance LOO predictions

Instance	Actual	DT Predicted	DT	NB Predicted	NB
D1	No	No	OK	Yes	X
D2	No	No	OK	No	OK
D3	Yes	Yes	OK	Yes	OK
D4	Yes	No	X	No	X
D5	Yes	No	X	Yes	OK
D6	No	Yes	X	Yes	X
D7	Yes	Yes	OK	Yes	OK
D8	No	No	OK	Yes	X
D9	Yes	Yes	OK	Yes	OK
D10	Yes	Yes	OK	Yes	OK
D11	Yes	No	X	No	X
D12	Yes	No	X	Yes	OK
D13	Yes	Yes	OK	Yes	OK
D14	No	Yes	X	Yes	X

Decision Tree vs Naive Bayes
Leave-One-Out CV Accuracy



Naive Bayes Probability Tables (PlayTennis)

P(feature | Play=Yes)

	Count	P(. Play=Yes)
Outlook = Overcast	4/9	0.444
Outlook = Rain	3/9	0.333
Outlook = Sunny	2/9	0.222
Temperature = Cool	3/9	0.333
Temperature = Hot	2/9	0.222
Temperature = Mild	4/9	0.444
Humidity = High	3/9	0.333
Humidity = Normal	6/9	0.667
Wind = Strong	3/9	0.333
Wind = Weak	6/9	0.667

P(feature | Play=No)

	Count	P(. Play=No)
Outlook = Overcast	0/5	0.000
Outlook = Rain	2/5	0.400
Outlook = Sunny	3/5	0.600
Temperature = Cool	1/5	0.200
Temperature = Hot	2/5	0.400
Temperature = Mild	2/5	0.400
Humidity = High	4/5	0.800
Humidity = Normal	1/5	0.200
Wind = Strong	3/5	0.600
Wind = Weak	2/5	0.400

Confusion Matrices -- PlayTennis LOO-CV

Decision Tree (LOO-CV)

Actual	No	3	2
	Yes	4	5
		No	Yes
		Predicted	

Naive Bayes (LOO-CV)

Actual	No	1	4
	Yes	2	7
		No	Yes
		Predicted	

Comparison and Discussion

Although both classifiers achieved the same LOO-CV accuracy of **57.1%**, they represent two fundamentally different approaches to classification and do not make the same mistakes.

The **Decision Tree** learns by recursively selecting the attribute with the highest information gain and constructing a sequence of rule-based splits. It does not assume independence between features; instead, it captures structure directly from the data through hierarchical if/else decisions. In contrast, **Naive Bayes** estimates class probabilities using Bayes' theorem under the assumption that the features are conditionally independent given the class label. It learns a set of probability tables from the training data and combines them multiplicatively at prediction time.

The tie in accuracy is therefore informative rather than trivial. In the PlayTennis dataset, some decisions depend on interactions between attributes rather than on individual attributes in isolation. For example, the combination **Sunny + High Humidity** is associated with class **No**, even though neither attribute alone is sufficient to determine the label. The Decision Tree captures such interactions naturally through successive splits, whereas Naive Bayes can only approximate them through independent probability estimates. This explains why the two classifiers achieve the same overall accuracy while still disagreeing on several specific instances, including D1, D4, D5, D8, D11, D12, and D14.

Dimension	Decision Tree	Naive Bayes
Main idea	Builds a tree of if/else decisions using information gain	Uses probabilities to predict the most likely class
How it handles features	Can capture combinations of attributes through successive splits	Treats attributes as independent given the class
Interpretability	High: easy to follow as decision rules	Moderate: based on probability tables rather than explicit rules
Behaviour on small datasets	Can fit the training data very closely and may overfit	Usually simpler and less sensitive to small variations
Smoothing	Not applicable	No smoothing was used here (alpha = 0)
Works well when	The class depends on combinations of attribute values	The attributes provide useful evidence individually

Part C - UCI Iris Dataset Experiment

Dataset Description

The UCI Iris dataset contains 150 flower samples equally divided among three classes: Iris setosa, Iris versicolor, and Iris virginica. Each sample is described by four continuous features measured in centimetres: sepal length, sepal width, petal length, and petal width.

Property	Value
Instances	150
Features	4 (sepal length, sepal width, petal length, petal width)
Classes	3 - Iris setosa, Iris versicolor, Iris virginica
Class balance	Perfectly balanced: 50 instances per class

Features statistics

Feature	Mean (cm)	Std Dev	Min	Max
Sepal length	5.84	0.83	4.3	7.9
Sepal width	3.06	0.43	2.0	4.4
Petal length	3.76	1.76	1.0	6.9
Petal width	1.20	0.76	0.1	2.5

A Decision Tree classifier was trained using the Gini impurity criterion. The maximum tree depth was limited to **4** in order to keep the model simple and reduce the risk of overfitting. This allows the tree to focus on the most informative splits instead of growing unnecessary branches, while still providing enough capacity to separate the three Iris classes effectively.

The dataset was divided into **120 training instances (80%)** and **30 test instances (20%)** using **stratified sampling** (stratify=y). Stratification preserves the class distribution in both sets, ensuring that each Iris class is represented fairly in the training and test data. This makes the evaluation more reliable than a random split that could accidentally create class imbalance.

Results

- **Test Accuracy:** 93.3%
- **Training Instances:** 120
- **Test Instances:** 30

Evaluation Metrics

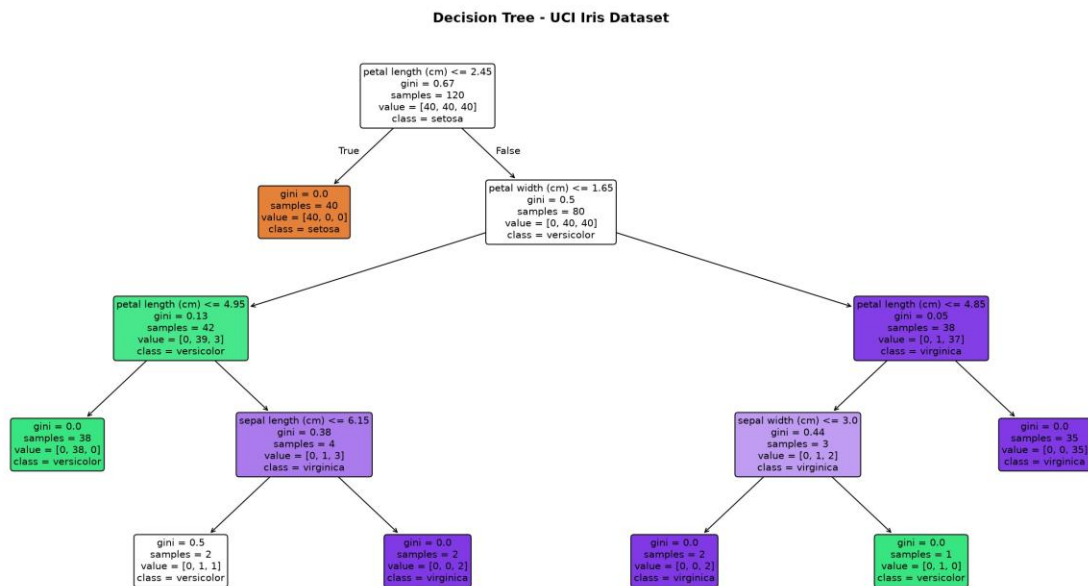
Class	Precision	Recall	F1-Score	Support
Setosa	1.00	1.00	1.00	10
Versicolor	0.90	0.90	0.90	10
Virginica	0.90	0.90	0.90	10
Macro average	0.93	0.93	0.93	30

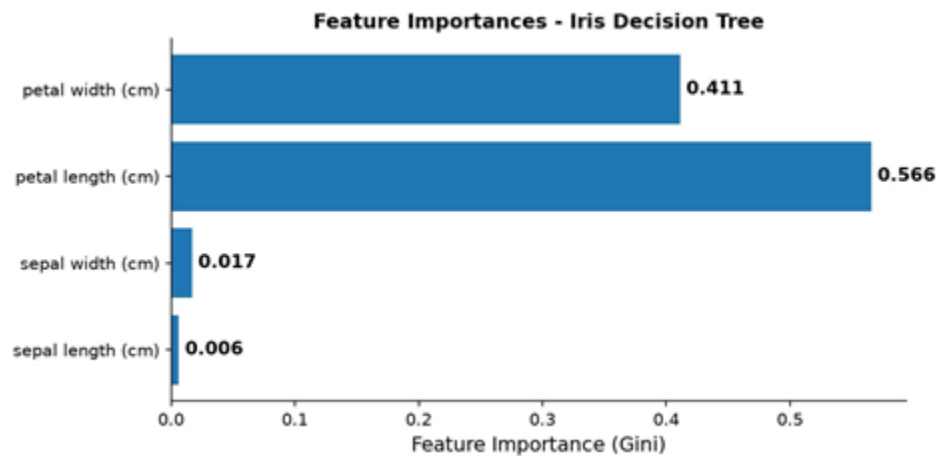
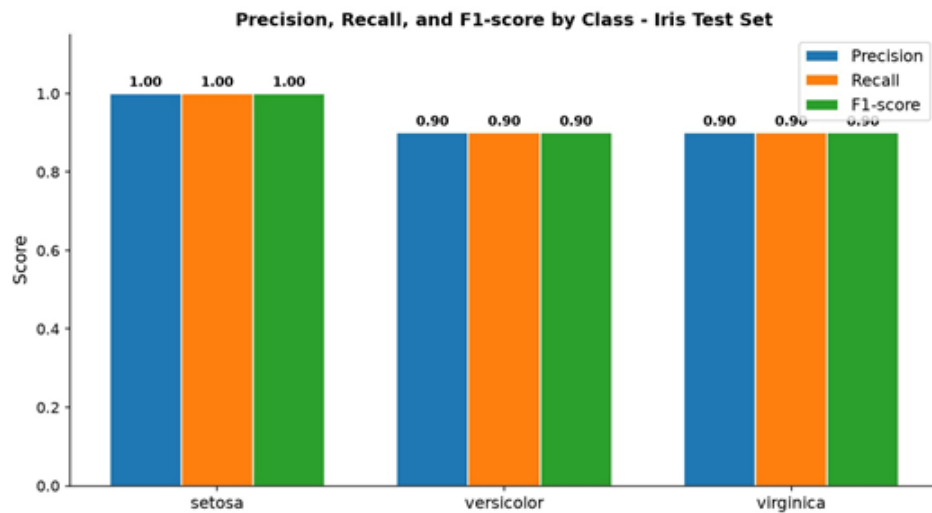
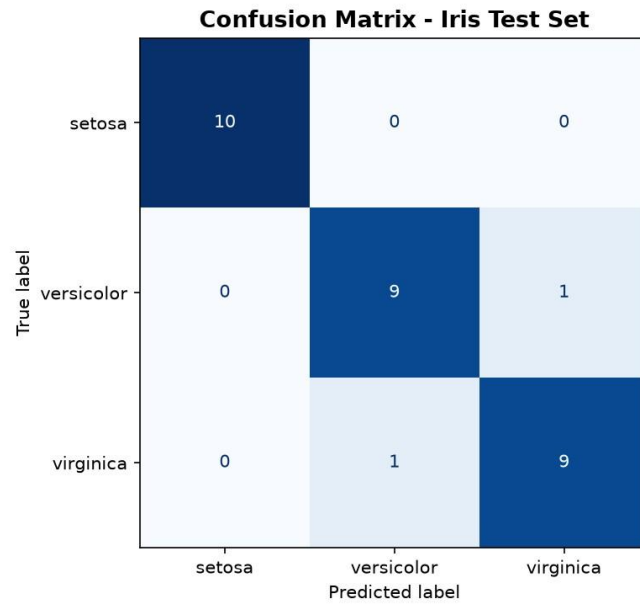
Confusion Matrix

Actual \ Predicted	Setosa	Versicolor	Virginica
Setosa	10	0	0
Versicolor	0	9	1
Virginica	0	1	9

Feature Importances

Feature	Gini Importance
sepal length (cm)	0.0063
sepal width (cm)	0.0169
petal length (cm)	0.5656
petal width (cm)	0.4112





Discussion

The feature importance scores show that petal length and petal width are the most useful features in the Iris dataset, while the sepal measurements contribute very little. This is because Iris setosa has much smaller petals than the other two classes, so a simple split on petal measurements can separate it very clearly. In contrast, sepal measurements overlap more across the classes, so they provide less useful information to the tree.

The confusion matrix shows only two classification errors, both between versicolor and virginica. This suggests that these two classes are less clearly separated than setosa, especially in the petal measurements near the class boundary. Limiting the tree depth to 4 helps keep the model simple and reduces the risk of overfitting, while still giving strong performance on the test set (93.3% accuracy).

Conclusion

This project implemented and evaluated machine learning classification methods on both the PlayTennis and UCI Iris datasets. In Part A, the ID3 algorithm was implemented from scratch and successfully reproduced the expected multi-way decision tree for PlayTennis using entropy and information gain. In Part B, Decision Tree and Naive Bayes were compared using Leave-One-Out Cross-Validation, showing that the two classifiers can achieve the same overall accuracy while still making different errors because they rely on different learning assumptions. In Part C, a Decision Tree classifier achieved strong performance on the UCI Iris dataset, with petal features providing most of the useful class-separating information.

Overall, the project demonstrates the practical differences between decision-tree learning and probabilistic classification, while also highlighting the importance of evaluation methodology, feature behaviour, and model interpretability in supervised learning tasks.